CSCI 49900: Capstone

Group 8: Alex Haskel, James Lacoste, Kailun You

10/14/2026



Overview

- Our project is an interactive platform to demonstrate the mechanics of large language model inference optimization and to allow for the exploration of model behavior through internal interventions.
- The website will allow: Prompt → baseline + configurable experimental run → side-by-side outputs + metrics.
- The project requires writing code to optimize a model, and creating an interactive platform in which to test model behavior and performance.
- Alex: Backend + interventions, James: Backend + inference optimization, Kailun: Frontend.



Backend Development (Alex)

Refined backend generation functions:

- Created new backend function to manage model intervention logic and text generation

```
def run_generation(model, tokenizer, prompt: str, request:ExperimentConfiguration):
    generation_settings = request.generation
    intervention_settings = request.intervention
    handles = []
```

- Required learning about try/finally blocks

```
finally:
    for handle in handles:
        handle.remove()

return model_response
```

Backend Development (Alex)

Updated API endpoint

- API has to reflect current changes in backend functions

```python
@app.post("/generate", response_model=GenerateResponse)
def generate(request:GenerateRequest):

    prompt = request.prompt
    configuration_a = request.configuration_a
    configuration_b = request.configuration_b
    generated_text_a = run_generation(model, tokenizer, prompt, configuration_a)
    generated_text_b = run_generation(model, tokenizer, prompt, configuration_b)
```

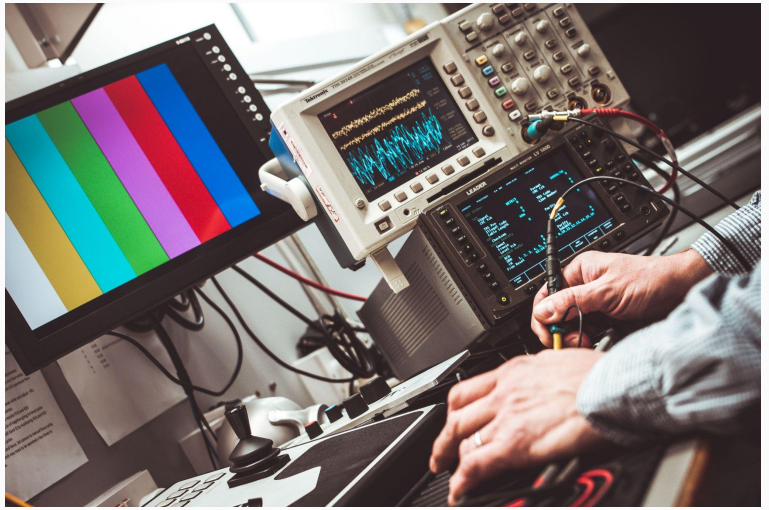
Tested Interventions in API (Alex)

```
{
  "prompt": "What is the secret of life?",
  "configuration_a": {
    "generation": {
      "max_new_tokens": 100,
      "do_sample": false,
      "temperature": 0.8,
      "top_k": 50,
      "top_p": 0.95,
      "repetition_penalty": 1.3,
      "enable_thinking": true
    }
  },
  "configuration_b": {
    "generation": {
      "max_new_tokens": 100,
      "do_sample": false,
      "temperature": 0.8,
      "top_k": 50,
      "top_p": 0.95,
      "repetition_penalty": 1.3,
      "enable_thinking": true
    },
    "intervention": {
      "type": "mlp",
      "layers": [0,2,4,6,8,10,12,14],
      "proportion": 0.65,
      "seed": 42
    }
  }
}
```

Execute

```
{
  "prompt": "What is the secret of life?",
  "configuration_a": {
    "generation": {
      "max_new_tokens": 100,
      "do_sample": false,
      "temperature": 0.8,
      "top_k": 50,
      "top_p": 0.95,
      "repetition_penalty": 1.3,
      "enable_thinking": true
    }
  },
  "configuration_b": {
    "generation": {
      "max_new_tokens": 100,
      "do_sample": false,
      "temperature": 0.8,
      "top_k": 50,
      "top_p": 0.95,
      "repetition_penalty": 1.3,
      "enable_thinking": true
    },
    "intervention": {
      "type": "attention",
      "layers": [0,2,4,6,8],
      "head_idxs": [0,1,2,3,4,5,6,7]
    }
  }
}
```

Execute



```
Model from_pretrained: 18.16s
Total load_model: 22.15s
Startup through model loading: 46.56s
INFO:     Started server process [3627]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:59438 - "GET / HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:59434 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:59434 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:42602 - "POST /generate HTTP/1.1" 200 OK
```

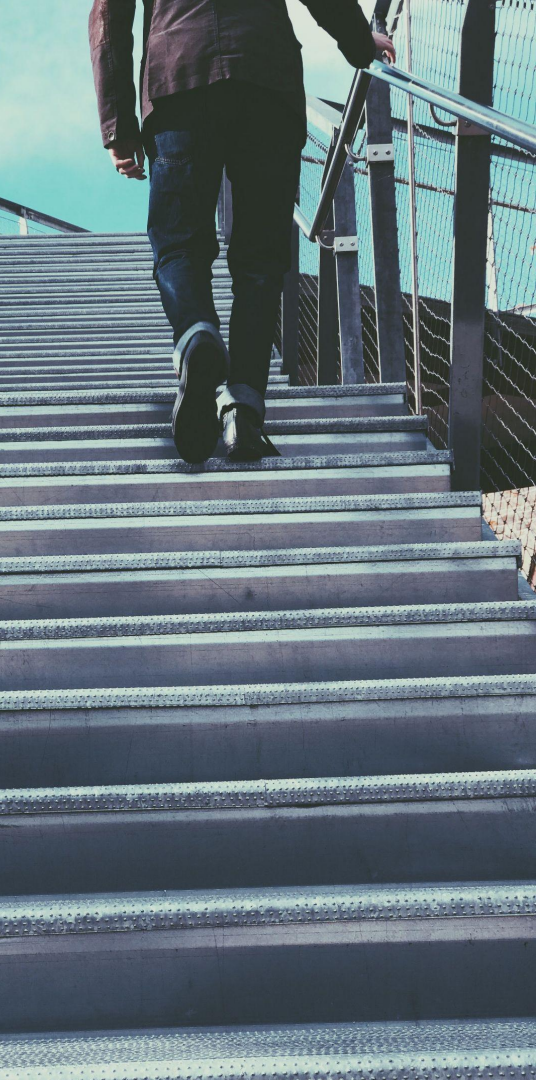
Generated JSON samples (Alex)

- Generated JSON file samples of generation requests and responses to help guide frontend design



JSON_examples

- request_attention_intervention.json
- request_mlp_intervention.json
- request_no_intervention.json
- successful_response_sample.json
- validation_error_generic.json
- validation_error_top_p_sample.json



Next Steps (Alex)

- Basic mechanics work
- Integrate back end and front end
- May modify code to allow for continuous chat
- Also may polish code to end text responses more gracefully
- Add other new features

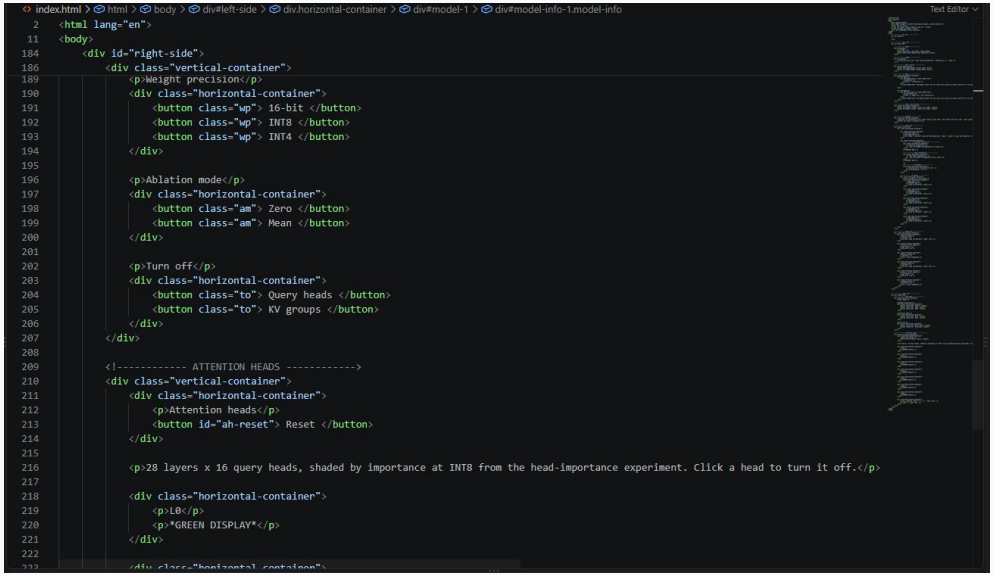
Kailun You

Responsibility: Frontend

Currently…

01

Website Main Functions and Display



Kailun You

Responsibility: Frontend

Planning...

01

Changes to the website appearance and simple styling to make the website more convenient

02

Using JS to register id and connect it to custom dummy files to display

Benchmarking

This week I focused on starting the **benchmark harness:**

- GPU timing with CUDA events: warmup, then the median and interquartile range over at least 50 runs
- clock, temperature and power should be logged roughly every 100 ms through nvidia management library
- a measured peak bandwidth test to provide the "100%" that kernel efficiency is measured against
- L2 flushing

Overview of Benchmarking

During decoding, Qwen3-0.6B produces one token at a time. For each token, every decoder layer (all 28 of them) runs seven linear layers on a single vector

Every weight is read from memory once and used for exactly one multiply add. (Dot product) That's about **1 FLOP per byte** read for FP16 weights.

Overview of Benchmarking

My GPU has 20 SMs (streaming multiprocessors) with 128 FP32 cores each, roughly 12.8 TFLOP/s of FP32 alone, against 183 GB/s of measured memory bandwidth

So it could do about 70 FLOPs per byte it reads, GEMV uses 1, which means the arithmetic units sit idle almost all the time, waiting for data

1. How fast can the GPU read memory at all? (in theory)
2. How close does a given kernel come to that?

Writing a kernel that just reads from the GPU

```cpp
__device__ __forceinline__ uint32_t xor4(uint4 v) { return v.x ^ v.y ^ v.z ^ v.w; }

__global__ void stream_read_kernel(const uint4* __restrict__ vec, int64_t n_vec,
                                   const uint8_t* __restrict__ tail, int64_t n_tail,
                                   uint32_t* __restrict__ partials) {
    const int64_t stride = static_cast<int64_t>(gridDim.x) * blockDim.x;
    int64_t i = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    uint32_t acc = 0;
    for (; i + 3 * stride < n_vec; i += 4 * stride) {
        const uint4 a = vec[i];
        const uint4 b = vec[i + stride];
        const uint4 c = vec[i + 2 * stride];
        const uint4 d = vec[i + 3 * stride];
        acc ^= xor4(a) ^ xor4(b) ^ xor4(c) ^ xor4(d);
    }
    for (; i < n_vec; i += stride) acc ^= xor4(vec[i]);

    // Bytes after the last full vector, as little-endian 32-bit words padded with zeros.
    if (blockIdx.x == 0) {
        for (int64_t j = threadIdx.x; j < n_tail; j += blockDim.x)
            acc ^= static_cast<uint32_t>(tail[j]) << (8 * (j % 4));
    }

    for (int offset = 16; offset > 0; offset /= 2) acc ^= __shfl_xor_sync(0xffffffff, acc, offset);
    __shared__ uint32_t warp_acc[32];
    const int lane = threadIdx.x % 32;
    const int warp = threadIdx.x / 32;
    if (lane == 0) warp_acc[warp] = acc;
    __syncthreads();
    if (warp == 0) {
        acc = lane < static_cast<int>((blockDim.x + 31) / 32) ? warp_acc[lane] : 0;
        for (int offset = 16; offset > 0; offset /= 2) acc ^= __shfl_xor_sync(0xffffffff, acc, offset);
        if (lane == 0) partials[blockIdx.x] = acc;
    }
}
```

Next Steps

- I need to continue building the kernel and move on to CUDA event markers. When the GPU reaches it, a timestamp will be written
- When the harness is ready, try to compare the two models
- If I have time, try using the existing backend endpoints to connect to the quantized model